Constitutional Software Analysis (CSoftA): A Governance-Based Classification Method for Software Systems

A Structural Theory of Authority, Verification, Enforcement, and Reachability in Software Systems

The log remembers. The policy declares. The constitution determines whether the transition can occur.

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

Software systems increasingly mediate authority. Access decisions, deployments, infrastructure modifications, model promotions, service communications, and operational workflows are governed by an expanding ecosystem of policies, credentials, approval mechanisms, and control systems.

Yet software engineering lacks a rigorous method for distinguishing governance that constrains state transitions from governance that merely observes them.

This paper introduces **Constitutional Software Analysis (CSoftA)**, a structural methodology for classifying governance according to how authority verification participates in the reachability of governed state transitions. The method formalizes authority verification as a prerequisite for governed state transitions and classifies governance into three constitutional states:

- **ACTIVE Governance** - authority verification is a necessary and non-bypassable precondition for reachability of a governed state transition.
- **CRYSTALLIZED Governance** - authority or authority verification is observed, recorded, or reconstructed but is not required for reachability of a governed state transition.
- **ABSENT Governance** - authority verification is neither required nor meaningfully recorded.

CSoftA evaluates governance at explicit authority boundaries rather than treating software systems as monolithic entities. Applied across eighty software systems spanning cloud infrastructure, identity management, service meshes, Kubernetes ecosystems, CI/CD platforms, infrastructure-as-code frameworks, observability systems, databases, messaging platforms, and machine learning tooling, the

method reveals a recurring distinction between constitutive governance and observational governance.

The central finding is that many systems commonly described as governance systems primarily preserve evidence of authority rather than requiring authority verification for state-transition reachability. This distinction provides a reproducible framework for analyzing software governance, comparing heterogeneous systems, identifying enforcement structures, and locating where authority actually binds within modern software architectures.

---

1. Introduction

Software engineering possesses mature theories of reliability, scalability, security, and performance.

It lacks an equivalent theory of authority.

Most governance discussions implicitly assume that policies, controls, approvals, logs, monitoring systems, and compliance artifacts belong to a single category. They do not.

A system that records a violation after it occurs is structurally different from a system that prevents the violation from occurring.

A system that recommends compliance is structurally different from a system that requires compliance.

A system that documents authority is structurally different from a system that enforces authority.

Constitutional Software Analysis begins from a simple observation:

> Governance is not defined by what a system says. Governance is defined by what a system requires.

The central question is therefore:

> Is the governed state transition reachable without successful authority verification?

Everything else follows from the answer.

---

2. Formal Foundations

2.1 State Transitions

A state transition is a mapping from system state S to system state S' induced by an attempted transition.

A state transition is reachable if there exists at least one valid transition path from S to S' under defined system and governance constraints.

Examples include:

- Anonymous user -> authenticated user
- Undeployed service -> deployed service
- Unauthorized request -> authorized request
- Draft model -> production model
- Unprovisioned infrastructure -> provisioned infrastructure

Governance is evaluated with respect to whether a particular state transition remains reachable.

A state transition is considered governed only if authority influences reachability of the transition itself rather than merely influencing observation of the transition after completion.

CSoftA may therefore be interpreted as a reachability analysis over constrained state transitions.

Reachability claims should be supported by observable system behavior, such as successful or failed transition attempts under controlled conditions.

Distinct state transitions that share identical reachability conditions may be analyzed as a single equivalence class.

Unless otherwise specified, all references to "the transition" refer to the governed state transition under analysis.

2.2 Constitutional Reachability Constraint

A state transition is **constitutionally constrained** if it is unreachable without satisfying authority verification.

This is the core property CSoftA evaluates.

A system may possess governance artifacts, policies, logs, approvals, or documentation, but unless those artifacts constrain reachability, they do not constitute ACTIVE governance.

2.3 Authority

Authority is a capability, credential, condition, or proof required to permit a governed state transition.

Authority answers:

> Who or what may cause this transition to occur?

Examples include:

- Authentication credentials
- Certificates
- Approval signatures
- Capability tokens
- Admission policies
- Cryptographic proofs
- Delegated trust relationships

Authority is not identity.

Identity answers who.

Authority answers whether.

2.4 Authority Possession and Authority Verification

These concepts must be distinguished.

Authority Possession The actor holds a credential, capability, proof, or authorization artifact.

Authority Verification The system validates and accepts that artifact.

The distinction is critical.

A credential may exist without being checked.

An authorization may be granted without being enforced.

A verification process may execute without influencing system behavior.

ACTIVE governance therefore depends not merely upon authority verification, but upon verification success being required for the governed state transition to become reachable.

Authority verification is evaluated over both state and time; validity must hold at the moment the transition becomes reachable.

Stale, replayed, expired, or time-of-check/time-of-use-broken verification that permits unauthorized reachability violates ACTIVE governance.

2.5 Governance

Governance is a mechanism that mediates access to authority.

Governance determines:

- Whether authority is required
- How authority is verified
- Under what conditions authority may be exercised

Governance answers:

> Under what conditions may authority be exercised?

2.6 Enforcement

Enforcement ensures governance conditions are actually required.

Governance defines admissibility.

Enforcement makes admissibility consequential.

ACTIVE governance therefore requires enforcement.

Governance without enforcement cannot bind state transitions.

Verification outcomes must deterministically and exclusively constrain reachability of the governed state transition.

A governance mechanism that evaluates authority but permits identical reachability regardless of verification outcome does not provide constitutional enforcement.

Partial or conditional enforcement that allows the governed state transition to become reachable without full verification success does not constitute ACTIVE governance.

Governance is not ACTIVE if enforcement can be predictably or systematically avoided under normal or degraded operating conditions.

2.7 State Transition Attempts

A state transition attempt is an effort to induce a transition from S to S'.

Execution is therefore treated as an attempted state transition rather than as a separate primitive concept.

2.8 Constitutional Principle

Governance is only constitutionally relevant where authority participates in constraining state-transition reachability.

Policies, documentation, organizational intent, compliance declarations, and audit artifacts do not by themselves constitute binding governance.

---

3. Governance States

3.1 ACTIVE Governance

ACTIVE governance exists when authority verification is a necessary and non-bypassable precondition for a governed state transition.

Without successful verification, the transition cannot become reachable.

Without governance, the transition cannot become reachable.

ACTIVE governance is therefore constitutive of reachability.

Examples include:

- Certificate verification in mutual TLS
- IAM permission checks
- Admission controllers that block deployment
- Mandatory approval gates
- Cryptographic authorization requirements

Formal Test Governance is ACTIVE iff:

1. Authority verification is performed and applied.
2. Verification success is required for the transition to become reachable.
3. The transition remains unreachable when verification fails.
4. Verification validity holds at the time of transition.
5. Governance cannot be bypassed within the analyzed boundary.

Evidence preservation is orthogonal to ACTIVE governance.

A system may be fully governed while preserving little or no evidence of governance decisions.

3.2 CRYSTALLIZED Governance

CRYSTALLIZED governance preserves evidence of authority or authority verification but does not require authority verification for the transition.

The transition remains reachable.

Governance remains as memory.

Examples include:

- Audit logs
- Monitoring systems
- Compliance reports
- Telemetry platforms
- Observability stacks

CRYSTALLIZED governance is causally downstream of the transition's reachability.

It may explain, reconstruct, observe, investigate, or support remediation of the transition.

It cannot prevent the initial reachability of the governed state transition.

Formal Test Governance is CRYSTALLIZED iff:

1. The transition remains reachable without authority verification.
2. Governance records, reconstructs, or observes authority, authority verification, or transition behavior.
3. Governance has no causal influence on whether the transition becomes reachable.

3.3 ABSENT Governance

ABSENT governance exists when authority verification is neither required nor meaningfully recorded.

No binding authority exists at the analyzed boundary.

Formal Test Governance is ABSENT iff:

1. The transition remains reachable without authority verification.
2. No meaningful governance mechanism constrains the transition.
3. No meaningful governance mechanism preserves accountability.

---

4. Bypass Resistance

A governance mechanism that can be trivially circumvented does not provide constitutional enforcement.

ACTIVE governance therefore requires bypass resistance.

Bypass

A bypass is a transition path that reaches the governed state transition without satisfying governance requirements while remaining inside the analyzed boundary.

Examples include:

- Alternate APIs
- Privileged escape paths
- Disabled admission controllers
- Unauthenticated management interfaces

If any reachable path to the governed state transition does not require successful authority verification, the transition is considered reachable without governance.

A secure path existing somewhere in the system is not sufficient.

The question is whether all reachable paths are governed under the assumed system and threat constraints.

Boundary Leakage

Boundary leakage occurs when a transition path exits the analyzed boundary, avoids governance, and re-enters to complete the governed state transition.

Examples include:

- Direct datastore modification that bypasses an API governance layer
- Side-channel administration paths
- External automation operating outside a control plane

A governance mechanism is not bypass-resistant if practical leakage paths exist.

A leakage path is considered practical if it is accessible without violating assumed system constraints or requiring extraordinary capabilities beyond the threat model.

Governance cannot be considered ACTIVE when practically available paths permit the governed transition to be reached without successful authority verification.

The constitutional question is not:

> Does enforcement exist?

The constitutional question is:

> Is enforcement unavoidable?

---

5. Boundaries

CSoftA evaluates governance at explicit authority boundaries.

A boundary is the minimal interface at which a governed state transition is made reachable or rendered unreachable by authority verification.

Examples include:

Process Boundaries

- Syscalls
- Runtime APIs

- Plugin interfaces

Network Boundaries

- Service-to-service communication
- API gateways
- Load balancers

Control Plane Boundaries

- Deployment admission
- Infrastructure provisioning
- Policy engines

Organizational Boundaries

- Human approvals
- Change management processes
- Promotion workflows

Classifications are meaningful only relative to a fixed boundary definition.

The same system may be ACTIVE at one boundary and ABSENT at another.

This is expected.

---

6. Governance Composition

Real systems consist of multiple governance layers.

Classifications must therefore compose.

Rule 1: Upstream Constraint Inheritance

If a transition requires passage through an ACTIVE boundary, downstream transitions inherit that authority requirement.

ACTIVE constraints propagate.

Rule 2: Observation Cannot Upgrade Enforcement

CRYSTALLIZED governance cannot upgrade an ABSENT boundary.

Logging an unauthorized transition does not transform it into an authorized transition.

Observation cannot create enforcement.

Rule 3: Minimal Enforcement Cut Set

A system's effective governance is determined by the smallest set of ACTIVE boundaries whose removal permits the governed transition.

This set is the **Enforcement Cut Set**.

Procedure:

1. Identify all ACTIVE boundaries.
2. Hypothetically remove or disable each boundary.
3. Determine whether the governed transition remains reachable.
4. Identify the smallest subset whose removal permits the transition.

These boundaries constitute the actual enforcement structure.

Everything else is supportive infrastructure.

Rule 4: Composite Transitions

Many workflows consist of multiple intermediate state transitions.

Examples include:

- CI/CD pipelines
- Infrastructure provisioning workflows
- Machine learning deployment pipelines

Governance may apply to individual transitions or to the composite transition as a whole.

If a required intermediate transition is constrained by an ACTIVE boundary, the composite workflow inherits that constraint.

---

7. Governance Conservation

Authority does not disappear.

It moves.

When governance is removed from one boundary, authority is transferred elsewhere:

- Administrators
- Platform operators
- Infrastructure owners
- Credential issuers
- Vendors
- Users

Apparent absence of governance is often governance relocation.

Authority relocation can be identified by tracing which actor, system, or process must intervene to prevent, approve, or reverse the governed state transition.

The location of authority can also be identified by determining which component must be modified, controlled, or bypassed to prevent the transition.

One purpose of CSoftA is locating where authority ultimately binds after such transfers occur.

The analysis therefore concerns not merely the existence of authority, but its structural location.

---

8. Governance Latency

ACTIVE governance may operate at different temporal points.

Pre-Transition

Authority verified before the transition begins.

Examples:

- Authentication
- Deployment approval

Inline Authority evaluated during the transition.

Examples:

- Service mesh authorization
- Runtime policy engines

Continuous The transition remains interruptible.

Examples:

- Runtime containment
- Kill-switch architectures
- Active process termination

Governance timing affects operational behavior but does not alter constitutional classification.

Post-transition remediation does not constitute ACTIVE governance if the governed transition was reachable without successful authority verification.

Rollback, quarantine, revocation, and penalty mechanisms may reduce consequences without altering constitutional classification.

Partial Transitions

Distributed systems frequently produce partial transitions, side effects, and intermediate commitments.

If a transition produces externally observable effects without successful authority verification, governance is not ACTIVE for that transition regardless of whether the system later fails, rolls back, or compensates.

Authority verification must constrain the transition before prohibited effects become reachable.

---

9. Corpus Overview

The initial CSoftA corpus analyzed eighty software systems across sixteen research waves.

Domains included:

- Cloud Infrastructure
- Identity and Access Management
- Kubernetes Ecosystems
- Service Meshes
- Secrets Management
- Infrastructure as Code
- CI/CD Systems
- Observability Platforms
- Databases
- Messaging Systems
- AI/ML Platforms
- Security Tooling
- Configuration Management

Each system was evaluated through boundary analysis rather than feature analysis.

The result is a comparative governance corpus rather than a product review catalog.

---

10. Key Findings

10.1 Observational Governance Is More Common Than Constitutive Governance

Many systems marketed as governance platforms primarily preserve evidence.

They answer:

> What happened?

rather than:

> What transitions are permitted?

The distinction appears repeatedly across logging, monitoring, observability, and compliance tooling.

10.2 Credentials Are Constitutional Objects

One pattern appeared repeatedly across identity systems, service meshes, cloud infrastructure, and secrets management.

A credential functions as a portable proof of prior authorization. Transitions frequently reduce to verification of that proof.

This suggests a broader architectural principle: governance systems externalize authorization into verifiable artifacts. The resulting pattern may be described as **Authorization-Carrying Transitions** — a transition becomes reachable only when accompanied by verifiable proof of authorization.

This pattern appears across IAM systems, Vault, SPIFFE/SPIRE, mutual TLS, and certificate infrastructures. The convergence is notable and suggests a common architectural attractor for ACTIVE governance design.

Note: the term "proof-carrying code" exists in programming languages theory (Necula, 1997) for a distinct concept — static verification that code satisfies a safety policy. The pattern described here operates at runtime over authority artifacts rather than at compile time over program properties. The structural resonance is intentional; the domains are different.

10.3 Governance Recurses

Systems that preserve evidence become governance targets themselves.

Logging systems.

Telemetry systems.

SIEM platforms.

Observability stacks.

If these systems can be modified without constraint, accountability becomes unreliable.

Evidence preservation does not imply correctness, completeness, or integrity of the recorded data.

Governance therefore recurses.

The observer must also be governed.

10.4 Defaults Are Constitutional

The effective constitution of a system is defined by its default operational state and the minimal actions required to alter that state.

Capabilities matter.

Defaults matter more.

Users inherit defaults before they inherit expertise.

10.5 AI Governance Remains Structurally Underdeveloped

Modern machine learning platforms often provide sophisticated model development pipelines while lacking equivalent deployment governance.

The result is an authority gap between model creation and model transition into production.

As AI systems acquire operational authority, this gap becomes increasingly consequential.

---

11. Worked Example: Kubernetes Deployment

Consider the deployment path:

```
kubectl apply -> API Server -
> Admission Controller -> etcd -> Scheduler
```

Admission Controller Boundary

Policy evaluates.

Deployment may be denied.

If denial prevents deployment:

Classification: ACTIVE

Authority binds.

Audit Logging Boundary Deployment occurs.

The event is recorded.

Classification: CRYSTALLIZED

Authority is observed.

---

etcd Persistence Boundary Persistence itself does not determine authorization.

Its classification depends upon prior governance layers.

The example demonstrates a core CSoftA principle:

Multiple governance states may coexist within a single architectural path.

The analysis applies to boundaries, not products.

---

12. Adversarial Interpretation

The adversary is assumed to possess full knowledge of reachable system behavior and the ability to explore available transition paths within practical constraints.

The adversary is further assumed to seek minimal authority while maximizing reachability of a target transition.

The question remains:

> Can the governed state transition still be reached?

If the answer is yes, governance is not constitutionally ACTIVE regardless of policy intent, documentation quality, or compliance claims.

This perspective aligns governance analysis with security reasoning while preserving the framework's structural focus.

---

13. Counterexamples and Edge Cases

Feature Flags with Audit Logs

The transition remains reachable.

Changes are recorded.

Classification: CRYSTALLIZED.

Soft-Fail Admission Controllers

Policies evaluate.

Violations generate warnings.

The transition proceeds.

Classification: CRYSTALLIZED.

Disabled Enforcement by Default

Enforcement exists as capability.

The transition remains reachable without activation.

Classification: ABSENT in the default state.

Potential capability does not alter constitutional reality.

Required but Unverified Authority

Policy declares authority is required.

The system never verifies the authority claim.

The transition remains reachable.

Classification: ABSENT (Misconfigured Governance).

Policy intent is not enforcement.

Stale Verification

Authority was valid at an earlier time.

The credential or verification result is reused after authority is revoked, expired, or no longer applicable.

The transition remains reachable.

Classification: Non-ACTIVE (Bypassable Enforcement or ABSENT depending on boundary structure).

Past authority is not present authority.

---

14. Limitations

CSoftA evaluates governance structure rather than governance quality.

ACTIVE governance is not inherently desirable.

CRYSTALLIZED governance is not inherently weak.

The framework evaluates where authority binds.

It does not prescribe optimal governance arrangements.

Future work includes:

- Governance metrics
- Automated classification
- Governance topology visualization
- Constitutional software design patterns
- AI-agent governance analysis
- Enforcement cut-set tooling

Conclusion

Software increasingly functions as an authority system.

Yet software engineering still lacks a precise vocabulary for describing where authority actually binds.

Constitutional Software Analysis provides such a vocabulary.

The framework distinguishes between governance that constrains state-transition reachability, governance that preserves evidence of state transitions, and governance that is absent entirely.

Its central claim is straightforward:

Authority is not what systems declare.

Authority is what systems require, verify, and enforce to constrain reachability of state transitions.

To understand governance, we must stop asking what policies exist and begin asking which state transitions remain unreachable without successful authority verification.

The answer reveals the constitution hidden inside the software.

Appendix A: Governance Decision Table

- **Authority Verification Required:** Yes
  Authority Verification Performed and Applied: Yes
  Verification Success Required: Yes
  Evidence Preserved: Optional
  Practical Bypass Within Boundary: No
  Classification: ACTIVE
- **Authority Verification Required:** Yes
  Authority Verification Performed and Applied: Yes
  Verification Success Required: Yes
  Evidence Preserved: Optional
  Practical Bypass Within Boundary: Yes
  Classification: Bypassable Enforcement (Non-ACTIVE)
- **Authority Verification Required:** Yes
  Authority Verification Performed and Applied: No
  Verification Success Required: No
  Evidence Preserved: Optional
  Practical Bypass Within Boundary: N/A
  Classification: ABSENT (Misconfigured Governance)
- **Authority Verification Required:** No
  Authority Verification Performed and Applied: No
  Verification Success Required: No
  Evidence Preserved: Yes
  Practical Bypass Within Boundary: N/A
  Classification: CRYSTALLIZED
- **Authority Verification Required:** No
  Authority Verification Performed and Applied: No
  Verification Success Required: No
  Evidence Preserved: No
  Practical Bypass Within Boundary: N/A
  Classification: ABSENT

ACTIVE classification requires all of the following:

- Authority verification required
- Authority verification performed and applied
- Verification success required
- Verification valid at the time of transition
- No practical bypass path within the defined boundary

Evidence preservation is orthogonal to enforcement.

A system may be fully governed while preserving little or no evidence, and may preserve extensive evidence while governing no transitions.

Evidence preservation does not imply correctness, completeness, or integrity of the recorded data.

Appendix B: Constitutional Questions

1. What governed state transition is being analyzed?
2. What authority permits that transition?
3. Who possesses that authority?
4. How is authority verified?
5. Is verification success required?
6. Is verification valid at the time of transition?
7. Can the transition become reachable without successful authority verification?
8. Can governance be bypassed?
9. Do boundary leakage paths exist?
10. Are all reachable paths governed under the assumed system and threat constraints?
11. Is evidence preserved?
12. Is preserved evidence correct, complete, and integrity-protected?
13. Who governs the governor?
14. What is the enforcement cut set?

Corpus-Wide Reference Positioning

This paper argues that software systems increasingly mediate authority. Its nearest external comparison fields are governance, software, open sovereign. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counterpressure - not as evidence that they caused the Ableman framework or that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: A written constitution can legitimate domination as easily as constrain it unless amendment, participa-

tion, enforcement, and appeal remain effective in practice. Modularity, secure-development practice, and reproducibility reduce classes of failure but do not guarantee architectural coherence or user sovereignty. Source availability and reproducible builds improve inspectability but do not alone guarantee comprehensibility, practical exit, hardware control, or sustainable maintenance. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[**E1**] Elinor Ostrom. *Governing the Commons: The Evolution of Institutions for Collective Action.* Cambridge University Press, 1990. DOI: 10.1017/CBO9780511807763. https://doi.org/10.1017/CBO9780511807763 **Relationship:** Predecessor institutional account of rule-governed collective action and polycentric governance.

[**E2**] Wil Waluchow and Dimitrios Kyritsis. "Constitutionalism." *Stanford Encyclopedia of Philosophy*, substantive revision 2023. https://plato.stanford.edu/entries/constitutionalism/ **Relationship:** Adjacent legal-philosophical account of limited authority; included to expose differences between political and computational constitutionalism.

[**E3**] Lawrence Lessig. *Code: Version 2.0.* Basic Books, 2006. https://codev2.cc/ **Relationship:** Predecessor argument that software architecture regulates behavior; the paper's contribution may be to make such regulation inspectable and revisable.

[**E4**] OECD. *Recommendation of the Council on Artificial Intelligence.* OECD/LEGAL/0449, adopted 2019 and updated 2024. legalinstruments.oecd.org source **Relationship:** Adjacent policy standard for accountability, traceability, transparency, safety, and human-centered values.

[**E5**] David L. Parnas. "On the Criteria To Be Used in Decomposing Systems into Modules." *Communications of the ACM* 15(12), 1972, pp. 1053-1058. DOI: 10.1145/361598.361623. https://doi.org/10.1145/361598.361623 **Relationship:**

Foundational modularity treatment used to compare information hiding with explicit semantic and governance interfaces.

[**E6**] Ward Cunningham. "The WyCash Portfolio Management System." *OOPSLA '92 Experience Report*, 1992. DOI: 10.1145/157709.157715. https://doi.org/10.1145/157709.157715 **Relationship:** Original technical-debt metaphor used to distinguish implementation debt from semantic debt.

[**E7**] Karen Scarfone, Murugiah Souppaya, and Donna Dodson. *Secure Software Development Framework (SSDF) Version 1.1*. NIST SP 800-218, 2022. DOI: 10.6028/NIST.SP.800-218. https://doi.org/10.6028/NIST.SP.800-218 **Relationship:** Adjacent secure-development framework for preparation, protection, production, and vulnerability response.

Ableman corpus

[**A1**] Adam Ableman. *Ableman Constitutional Governance*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-79353AF163A4B0F1`;
source SHA-256
`FB0D9B2D3D54168FFE70B24308D1B2766D6D663ED24502D475E975939BE97C8B`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *Constitutional Theory*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-D2D97C4042FD281F`;
source SHA-256
`FACC256EDDE5A7D7EA18133CFC9C060CFB8C3811B0221E1969E62347F2BB69CF`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *Governance Engineering*. Ableman Corpus current edition; work identity `PAPER.PUB.18749870`;
source SHA-256
`372169F315D192733856B1F4A3F1038BBC73F57CF3ED9ACABAE72B965A69C8D0`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *Universal Governance Kernel (UGK)*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-1F9BFB2BF8370226`;
source SHA-256
`AA5106EA71E5DF120E141435C280DFC532715CA5DE0C75B8862BB7784ECCFACD`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *Legitimacy Without Self-Ratification*. Ableman Corpus current edition; work identity

AMCS-LOCAL-WORK-EF2382F9397335AC;
source SHA-256
0186A39C808F8FDDB5C8F42384DF1A26678928434BD50A0EAE8D4947390F68A2.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *Sovereign Software*. Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-24C6159B98023A9C;
source SHA-256
6AAFE79E438B980B75D1910A897F7BA716A10CF1187B68CB1F71405F36725A18.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *Semantic Fragmentation and Constitutional Structure in Software Systems*. Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-B307F65B51C503DE;
source SHA-256
9D2A9D79DE8BC3F2F8ABECA073E4D3607F7623F4368E63F8036081E39B4C75C3.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Editorial source notes retained from predecessor

Ableman, A. Constitutional Software Analysis Corpus. CSoftA-Canonical. https://github.com/AblemanMosaic/CSoftA-Canonical. 2026.

Ableman, A. Constitutional Computing Research Corpus. Constitutional Meaning Base (CMB). Independent research corpus. 2025–2026.

Ableman, A. Constitutional Governance Stack for Reasoning Systems. Ableman Research. 2025.

Ableman, A. Semantic Reactor Theory. Ableman Research. 2024. https://zenodo.org/records/20153069

Ableman, A. Ableman's Razor. Ableman Research. 2024. https://doi.org/10.5281/zenodo.18113139

Google. BeyondCorp: A New Approach to Enterprise Security. research.google source Risher, Oswald, and Ward

Lampson, B. Protection. Proceedings of the 5th Princeton Conference on Information Sciences and Systems. 1971. Reprinted in ACM Operating Systems Review, 8(1):18–24. 1974.

Necula, G. C. Proof-Carrying Code. Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL). 1997. pp. 106–119.

NIST. SP 800-207: Zero Trust Architecture. National Institute of Standards and Technology. 2020. https://doi.org/10.6028/NIST.SP.800-207

Ostrom, E. Governing the Commons: The Evolution of Institutions for Collective Action. Cambridge University Press. 1990.

Saltzer, J. H., and Schroeder, M. D. The Protection of Information in Computer Systems. Proceedings of the IEEE, 63(9):1278–1308. 1975.

SPIFFE Project. SPIFFE and SPIRE Documentation. https://spiffe.io/docs/. Accessed 2026.

HashiCorp. Vault Documentation. https://developer.hashicorp.com/vault/docs. Accessed 2026.

Kubernetes Project. Kubernetes Documentation. https://kubernetes.io/docs/. Accessed 2026.

Amazon Web Services. AWS IAM Documentation. https://docs.aws.amazon.com/iam/. Accessed 2026.

Amazon Web Services. AWS CodePipeline Documentation. https://docs.aws.amazon.com/codepipeline/. Accessed 2026.

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus
License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.20472196